

Course outline (Syllabus of the course) (Template – 1)

Course Code: CS2000		
Name of the Programme: B.Tech(H)		
Name of the Course: Design and Analysis of Algorithms		
Semester: III		
Course credit	No. of hours per week	Total no. of Teaching hours
04	2+1+2	75
Course Objectives:		
Develop proficiency in designing efficient algorithms to solve complex computational problems across diverse domains using techniques like divide-and-conquer, greedy methods, and backtracking.		
Gain the ability to evaluate and compare algorithm performance through rigorous analysis, including time and space complexity, to optimize solutions for practical applications.		
Master the implementation of fundamental algorithms, such as sorting, searching, and graph traversal, in a programming environment to build robust software solutions.		
Cultivate problem-solving skills by applying algorithmic strategies and data structures to real-world challenges, fostering innovation in areas like optimization, networking, and data processing.		

Syllabus:

Module - 1 Introduction to Algorithms, Basics of Searching and Sorting	No. of hours
	12
Introduction to Algorithms, Asymptotic notation and growth of functions, Analyzing complexity of algorithms, Recurrence relation, Substitution method, Recursion Tree Method, Master's Theorem, Brute force Design Technique : Linear Search, Naïve string-matching algorithm. Sorting methods: Bubble sort, Insertion Sort, and Selection Sort.	

Module - 2 Divide and Conquer & Decrease and Conquer	No. of hours
	8
Divide & Conquer: Merge Sort, Quick-Sort, Strassen's Matrix Multiplication Decrease & Conquer: Binary Search, Heap sort, Basics of graphs : Graph representation, DFS and BFS, Counting sort.	

Module - 3 Greedy Design Technique	No. of hours
	10

Elements of greedy strategy, Knapsack problem, Job sequencing with deadlines, Huffman encoding. Spanning trees, Minimum cost spanning trees: Kruskal's and Prim's algorithms; Single source shortest path algorithms, Dijkstra's Algorithms.

Module - 4 Dynamic Programming	No. of hours
	8
Elements of dynamic programming, illustration of use of dynamic programming with specific examples such as 0/1 Knapsack problem, Matrix chain multiplication, finding longest common subsequence, Bellman-Ford Algorithm, Floyd Warshal's algorithm.	

Module - 5 String Matching and NP Completeness	No. of hours
	7
Backtracking: N-Queens problem, Hamiltonian Cycle. Branch & Bound: Knapsack Problem String matching: Rabin-Karp algorithm, Knuth-Morris-Pratt algorithm. Introduction of NP-completeness: P, NP, NPC, NPH problems.	

Lab Programs		[30 Hours]
1	Program to implement Binary Search using Divide and Conquer	
2	Program to implement minimum and maximum element in given array using Divide and Conquer	
3	Program to implement Merge sort using Divide and Conquer	
5	Program to implement Quick sort	
6	Program to implement Heap sort	
7	Program to implement Graph Traversal: Breadth First Traversal	
8	Program to implement Graph Traversal: Depth First Traversal	
9	Program to implement Knapsack problem using Greedy method	
10	Program to implement Prim's algorithm using Greedy method	
11	Program to implement Kruskal's algorithm using Greedy method	
12	Program to implement All Pairs Shortest Path Using Dynamic Programming	

Course Outcomes: After completing the course, the students will be able to:	
CO1	Understand and analyze algorithms
CO2	Design efficient algorithms using Divide & Conquer and Decrease & Conquer techniques
CO3	Apply greedy & dynamic programming strategies to optimization problems
CO4	Explore advanced algorithm design techniques

Text books

1.	Cormen H. T., Leiserson E. C., Rivest L. R., and Stein C., Introduction to Algorithms, MIT Press (2009) 3rd ed. ISBN 978-0-262-53305-8
2.	Anany Leviton, “Introduction to the Design and Analysis of Algorithms”, 3rd Edition, ISBN 13: 978-0-13-231681-1.

Reference books

1.	Aho, Hopcraft, Ullman, “The Design and Analysis of Computer Algorithms”, Pearson [ISBN-10 : 0201000296, ISBN-13 : 978-0201000290]
2.	Dasgupta, Papadimitriou and Vazirani, “Algorithms”, TMH [ISBN-10 : 9780073523408, ISBN-13 : 978-0073523408]
3.	Complete YouTube Playlist (Hindi) by Shoeb Ahmad (Subject Teacher) Design and Analysis of Algorithms: https://shorturl.at/Ryz2L

Lesson plan (Template –2)

Name of the Programme:	B.Tech(H)	
Title of the Course:	Design and Analysis of Algorithms	
Course code	CS2000	
Semester:	III	
Names of the Course Instructor(s)	Prof. Shueb Ahmad, Prof. Kalpana Devi S, Prof. Shivi Sharma	
Course credit	No. of hours per week (2+1+2)	Total no. of Teaching hours
04	05 Hours	60 Hours

Sess ion	Module No.	Topic	Pedagogy /Activities	Pre-reading/refer ence
1.	1	Introduction to Algorithms	Interactive lecture	[Cormen] Ch. 3, [Levitin] Ch. 2
2.		Asymptotic notation	Interactive lecture	[Cormen] Ch. 3, [Levitin] Ch. 2
3.		growth of functions	Interactive lecture	[Cormen] Ch. 3, [Levitin] Ch. 2
4.		Lab 1: Program to implement Binary Search using Divide and Conquer	Hands-on using Hacker rank	[Levitin] Ch. 4, [Cormen] Ch. 2.3
5.		Analyzing complexity of algorithms	Interactive lecture	Cormen] Ch. 3, [Levitin] Ch. 2
6.		Recurrence relation, Substitution method	Interactive lecture	[Cormen] Ch. 4, [Levitin] Ch. 3
7.		Recursion Tree Method, Master's Theorem	Interactive lecture	Cormen] Ch. 4, [Levitin] Ch. 3
8.		Lab 2: Program to implement minimum and maximum element in given array using Divide and Conquer	Hands-on using Hacker rank	[Levitin] Ch. 4, [Cormen] Ch. 4.1
9.		Linear Search, Naïve string-matching algorithm	Interactive lecture	[Cormen] Ch. 32, [Levitin] Ch. 7
10.		Bubble sort, Insertion Sort	Interactive lecture	[Cormen] Ch. 2.1-2.3, [Levitin] Ch. 2
11.		Selection Sort.	Interactive lecture	[Cormen] Ch. 2.3, 7.1–7.4, [Levitin] Ch. 4

12.		Lab 3: Program to implement Merge sort using Divide and Conquer	Hands-on using Hacker rank	[Cormen] Ch. 2.3, 4; [Levitin] Ch. 4
13.	2	Divide & Conquer: Merge Sort	Interactive lecture	[Cormen] Ch. 2.3, 7.1–7.4, [Levitin] Ch. 4
14.		Quick-Sort	Interactive lecture	[Cormen] Ch. 2.3, 7.1–7.4, [Levitin] Ch. 4
15.		Strassen's Matrix Multiplication	Interactive lecture	[Cormen] Ch. 4.2, [Levitin] Ch. 5
16.		Lab 4: Program to implement Quick sort	Hands-on using Hacker rank	[Cormen] Ch. 7, [Levitin] Ch. 4
17.		Heap sort	Interactive lecture	[Cormen] Ch. 6, [Levitin] Ch. 7
18.		Basics of graphs : Graph representation	Interactive lecture	[Cormen] Ch. 22.1, [Levitin] Ch. 6
19.		DFS	Interactive lecture	[Cormen] Ch. 22.3, 22.2, [Levitin] Ch. 6
20.		Lab 5: Program to implement Heap sort	Interactive lecture	[Cormen] Ch. 6, [Levitin] Ch. 7
21.		BFS	Interactive lecture	[Cormen] Ch. 8.2, [Levitin] Ch. 2
22.		Counting sort	Interactive lecture	[Cormen] Ch. 8.2, [Levitin] Ch. 2
23.		Elements of greedy strategy	Interactive lecture	[Cormen] Ch. 16.2, [Levitin] Ch. 9
24.		Lab 6: Program to implement Graph Traversal: Breadth First Traversal	Hands-on using Hacker rank	[Cormen] Ch. 22.2, [Levitin] Ch. 6
25.	3	Knapsack problem	Interactive lecture	[Cormen] Ch. 16.2, [Levitin] Ch. 9
26.		Job sequencing with deadlines	Interactive lecture	[Levitin] Ch. 9
27.		Huffman encoding	Interactive lecture	[Cormen] Ch. 16.3, [Levitin] Ch. 9

28.		Lab 7: Program to implement Graph Traversal: Depth First Traversal	Hands-on using Hacker rank	[Cormen] Ch. 22.3, [Levitin] Ch. 6
29.		Spanning trees	Interactive lecture	[Cormen] Ch. 23, [Levitin] Ch. 9
30.		Minimum cost spanning trees	Interactive lecture	[Cormen] Ch. 23, [Levitin] Ch. 9
31.		Kruskal's algorithms	Interactive lecture	[Cormen] Ch. 23, [Levitin] Ch. 9
32.		Lab 8: Program to implement Knapsack problem using Greedy method	Hands-on using Hacker rank	[Cormen] Ch. 16.2, [Levitin] Ch. 9
33.		Single source shortest path algorithms,	Interactive lecture	[Cormen] Ch. 23, [Levitin] Ch. 9
34.		Dijkstra's Algorithms.	Interactive lecture	[Cormen] Ch. 24.3, [Levitin] Ch. 9
35.		Elements of dynamic programming	Interactive lecture	[Cormen] Ch. 15, [Levitin] Ch. 8
36.	4	Lab 9: Program to implement Prim's algorithm using Greedy method	Hands-on using Hacker rank	[Cormen] Ch. 23.2, [Levitin] Ch. 9
37.		0/1 Knapsack problem	Interactive lecture	[Cormen] Ch. 15.1, 15.2, [Levitin] Ch. 8
38.		Matrix chain multiplication	Interactive lecture	[Cormen] Ch. 15.1, 15.2, [Levitin] Ch. 8
39.		finding longest common subsequence	Interactive lecture	[Cormen] Ch. 15.4, [Levitin] Ch. 8
40.		Lab 10: Program to implement Kruskal's algorithm using Greedy method	Hands-on using Hacker rank	[Cormen] Ch. 23.2, [Levitin] Ch. 9
41.		Bellman-Ford Algorithm	Interactive lecture	[Cormen] Ch. 24.1, 25.2, [Levitin] Ch. 9
42.		Floyd Warshal's algorithm	Interactive lecture	[Cormen] Ch. 24.1, 25.2, [Levitin] Ch. 9
43.		Introduction to Backtracking	Interactive lecture	[Levitin] Ch. 11, [Cormen] (as exercise problems)
44.		Lab 11: Program to implement All Pairs Shortest Path Using Dynamic Programming	Hands-on using Hacker rank	[Cormen] Ch. 25.2, [Levitin] Ch. 8

45.		N-Queens problem	Interactive lecture	[Levitin] Ch. 11, [Cormen] (as exercise problems)
46.		Hamiltonian Cycle	Interactive lecture	[Levitin] Ch. 11, [Cormen] (as exercise problems)
47.		Introduction to Branch & Bound	Interactive lecture	[Levitin] Ch. 11
48.		Lab 12: Program to implement N-Queen's problem using Backtracking	Hands-on using Hacker rank	[Levitin] Ch. 11,
49.	5	Knapsack Problem	Interactive lecture	[Cormen] Ch. 15.1, 15.2, [Levitin] Ch. 8
50.		Rabin-Karp algorithm	Interactive lecture	[Cormen] Ch. 32.2, 32.4, [Levitin] Ch. 7
51.		Knuth-Morris-Pratt algorithm	Interactive lecture	[Cormen] Ch. 32.2, 32.4, [Levitin] Ch. 7
52.		Introduction of NP-completeness	Interactive lecture	[Cormen] Ch. 34, [Levitin] Ch. 12
53.		P, NP	Interactive lecture	[Cormen] Ch. 34, [Levitin] Ch. 12
54.		NPC, NPH problems.	Interactive lecture	[Cormen] Ch. 34, [Levitin] Ch. 12
55.				
56.				
57.				
58.				
59.				
60.				

Assessment and Evaluation

Continuous Internal Evaluation (CIE) :70%	
Components of CIE	Weightage of each component
CIE-1	

- Quiz Graded Component - Problem Solving / Programs	10 marks out of 70 10 marks out of 70
CIE-2 - Theory	25 marks out of 70
CIE-3 - Lab Experiment - Viva - Programming Assessment	4 marks out of 70 6 marks out of 70 15 marks out of 70

Semester End Exams (SEE): 30%	
Mode of Exam	Theory
Weightage of exam	30%

Course Outcomes- Programme Outcomes Matrix

Program Outcome Course Outcome	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2
CO 1 CO1: Design algorithms for addressing sub problems and tasks involving an algorithm component using appropriate design techniques (brute force, greedy, dynamic programming, etc.) as relevant, these tasks could	3	2	3		3	1						

	arise in the context of various applications such as engineering, Operations Research, e-commerce and so on.												
CO 2	CO2: Implement a variety of standard algorithms typically covered in an undergraduate course on Design and Analysis of Algorithms such as well known sorting and searching algorithms, graph related algorithms etc., in a high level language.	3	2	3		3							
CO 3	CO3: Analyze and be able to categorize the performance of different algorithms. This may include characterizing the performance of an algorithm using asymptotic notation, additionally,	3	2	3	2	2	1						

	the student should be able to identify the complexity class of well known problems.												
CO 4	CO4: Apply and implement learned algorithm design techniques and data structures to solve real world problems.	3	2	3	2	2	1	1	1				

Rubrics for evaluation of CIE (separate for each CIE component)

Total Marks		20
CIE-1 Components		Rubrics for Assessment
CIE-1	Quiz Graded Component 1	Quiz will be conducted for 10 marks. Each right answer carries one mark each. No negative marking for wrong answers.
	Problem Solving / Programs	Problem Solving / Programs will be conducted for 10 marks.

Total Marks		25
CIE-2 Components		Rubrics for Assessment
CIE-2	Theory	Test will be conducted for 25 marks and will be evaluated according to Scheme of Evaluation prepared by team of course faculty.

Total Marks		25
CIE-3 Components		Rubrics for Assessment
CIE-3	Practical Component	Lab Experiment+ Viva = 4+6 marks (Refer Table-3) Programming Assessment (Hacker rank) = 15 Marks

Table-3

Lab Write-up and Execution rubrics (Max: 4 marks)					
Sl No	Criteria	Measuring Methods	Excellent (2 Marks)	Good (1 Mark)	Poor (0 Marks)
1.	Understanding of problem statements. (2 Marks)	Observations	Student exhibits thorough understanding of requirements and applies suitable algorithm for the problem	Student has sufficient understanding of requirements and applies suitable algorithm for the problem.	Student does not have a clear understanding of requirements and is unable to apply suitable algorithm for the problem.

2.	Execution (2 Marks)	Observations	Student demonstrates the execution of the program with optimized code and shows performance efficiency. Appropriate validations with all test cases are handled.	Student demonstrates the execution of the program without optimization of the code and shows performance efficiency with only few test cases.	Student has not executed the program.
3.	Results and Documentation (2 Marks)	Observations	Documentation with appropriate comments and output is covered in data sheets and manual.	Documentation with only few comments and only few output cases is covered in data sheets and manual.	Documentation with no comments and no output cases is covered in data sheets and manual.
Viva Voce Rubrics (Max: 6 marks)					
Sl No	Criteria	Measuring Methods	Excellent (2 Marks)	Good (1 Mark)	Poor (0 Marks)

1.	Conceptual Understanding of Design Techniques (2 Marks)	Viva Voce	Explains thoroughly the algorithm and the data structure used along with related concepts.	Adequately explains the algorithms and data structures with related concepts	Unable to explain the algorithm and data structure.
2.	Use of appropriate Design Techniques (2 Mark)	Viva Voce	Insightful explanation of appropriate design techniques for the given problem to derive solution.	Sufficiently explains the use of appropriate design techniques for the given problem to derive solution.	Unable to explain the design techniques for the given problem.
	Communication of Concepts (2 Mark)	Viva Voce	Communicates the concept used in problem solving well.	Sufficiently communicates the concepts used in problem solving.	Unable to communicate the concepts used in problem solving.

Signature of the Course Faculty